

1

Lab

PHỤC VỤ MỤC ĐÍCH GIÁO DỤC
FOR EDUCATIONAL PURPOSE ONLY

ÔN TẬP NGÔN NGỮ ASSEMBLY & CHÈN MÃ VÀO TẬP TIN PE

Thực hành môn Cơ chế hoạt động của mã độc

Tháng 3/2026

Lưu hành nội bộ

<Nghiêm cấm đăng tải trên internet dưới mọi hình thức>

A. TỔNG QUAN

A.1 Mục tiêu

- Tìm hiểu ngôn ngữ assembly (hợp ngữ)
- Thực hiện một số chương trình mẫu trên IDE online
- Hiểu được cơ chế chèn mã vào file PE của viruses.
- Thực hiện chèn một đoạn mã đơn giản để hiển thị một message box khi mở file notepad.exe (32bit)

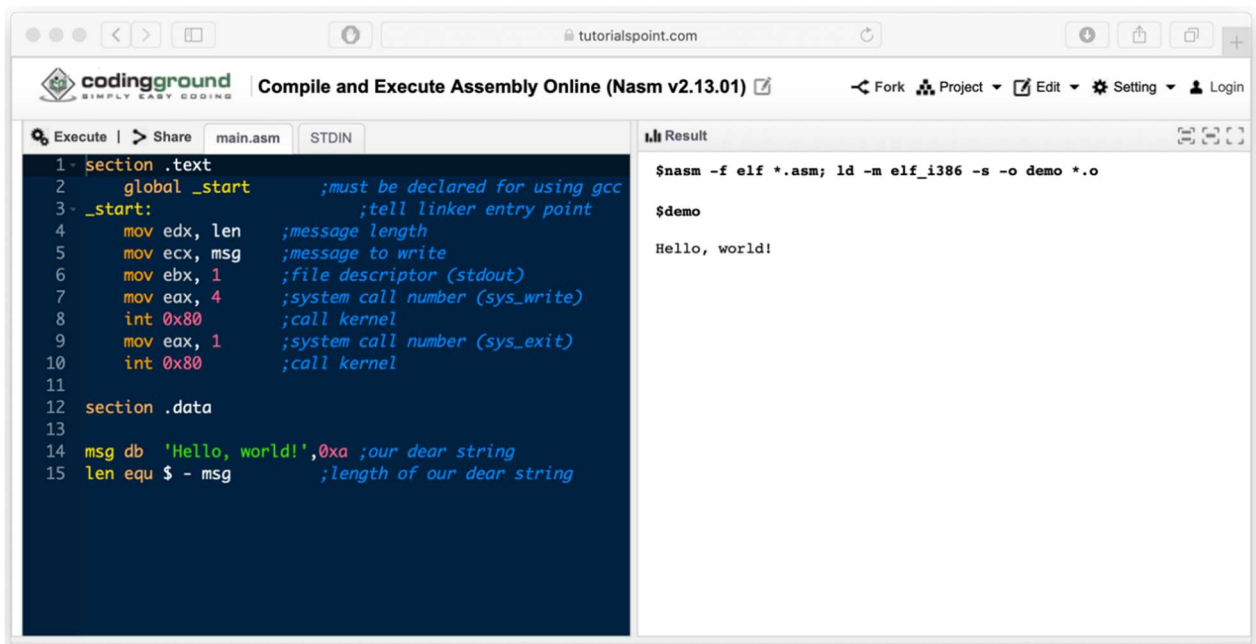
A.2 Môi trường

A.2.1 Cho phần thực hành code assembly

Sinh viên lựa chọn một trong hai cách sau để thiết lập môi trường thực hành.

a) Chạy code assembly trên IDE online

1. Truy cập http://www.tutorialspoint.com/compile_assembly_online.php
2. Nhập mã assembly vào cửa sổ bên trái.
3. Nhấn vào **Execute** để chạy tập tin thực thi.



b) Chạy code assembly trên Linux NASM

1. Tạo tập tin nguồn **test.asm**, thêm nội dung mã nguồn assembly vào tập tin.
2. Biên dịch tập tin với các lệnh:

```
nasm -f elf test.asm # để biên dịch code ra tập tin test.o
```

```
ld -m elf_i386 test.o -o test # tạo tập tin thực thi test
```

3. Chạy tập tin thực thi

```
./test
```

A.2.2 Cho phân thực hành chèn mã thực thi vào file PE

- Cài đặt [CFF Explorer](#) để xem và chỉnh sửa các tham số trong file PE.
- Cài đặt [HxD](#) để đọc và sửa nội dung file PE.
- Cài đặt **IDA Pro** để kiểm tra mã hợp ngữ của chương trình muốn chèn.

(Nên sử dụng máy ảo Windows để thực hành)

B. KIẾN THỨC NỀN TẢNG

B.1 Tổng quan về hợp ngữ

Hợp ngữ là một ngôn ngữ lập trình cấp thấp đối với một máy tính hoặc thiết bị lập trình khác đặc trưng cho một kiến trúc máy tính cụ thể trái ngược với hầu hết các ngôn ngữ lập trình cấp cao. Hợp ngữ được chuyển thành mã máy thực thi bằng một số trình biên dịch như NASM, MASM... Trong phạm vi bài thực hành, sinh viên sử dụng hợp ngữ x86 với định dạng Intel.

Một ví dụ về hợp ngữ chương trình xuất ra chữ "Hello, world!":

```
section .text
    global _start          ;must be declared for linker (ld)
_start:                   ;tells linker entry point
    mov     edx,len        ;message length
    mov     ecx,msg        ;message to write
    mov     ebx,1          ;file descriptor (stdout)
    mov     eax,4          ;system call number (sys_write)
    int     0x80           ;call kernel

    mov     eax,1          ;system call number (sys_exit)
    int     0x80           ;call kernel

section .data
msg db 'Hello, world!', 0xa ;our dear string
len equ $ - msg           ;length of our dear string
```

Hợp ngữ là gì?

Mỗi máy tính cá nhân đều có một bộ vi xử lý (CPU), đảm nhiệm các chức năng số học, logic và điều khiển hoạt động của hệ thống.

Mỗi dòng vi xử lý được thiết kế với một tập lệnh (instruction set) riêng để thực hiện các thao tác khác nhau, ví dụ như nhận dữ liệu từ bàn phím, hiển thị thông tin lên màn hình, hoặc điều khiển các thiết bị ngoại vi. Các tập lệnh này chính là ngôn ngữ máy, được biểu diễn trực tiếp bằng các chuỗi nhị phân (0 và 1) mà bộ xử lý có thể hiểu và thực thi.

Tuy nhiên, ngôn ngữ máy có cú pháp phức tạp, khó ghi nhớ và khó áp dụng trong phát triển phần mềm. Để khắc phục hạn chế này, người ta xây dựng ngôn ngữ hợp ngữ (assembly language) – một ngôn ngữ lập trình bậc thấp, sử dụng ký hiệu mnemonics thay thế cho các chuỗi nhị phân. Hợp ngữ được thiết kế gắn liền với một kiến trúc vi xử lý cụ

thể, giúp con người có thể lập trình ở mức gần với phần cứng nhưng dễ hiểu và dễ quản lý hơn so với ngôn ngữ máy.

Ưu điểm của hợp ngữ

Hiểu hợp ngữ có thể giúp nhận thức được:

- Cách các chương trình giao tiếp với các hệ điều hành, bộ xử lý, và BIOS;
- Cách dữ liệu được biểu diễn trong bộ nhớ và các thiết bị bên ngoài khác;
- Cách các bộ vi xử lý truy cập và thực hiện các lệnh;
- Cách các lệnh truy cập và xử lý dữ liệu;
- Cách một chương trình truy cập các thiết bị bên ngoài.

Những lợi ích khác của việc sử dụng hợp ngữ là:

- Sử dụng ít bộ nhớ và thời gian thực hiện;
- Cho phép phần cứng cụ thể hóa công việc phức tạp một cách dễ dàng hơn;
- Phù hợp cho công việc yêu cầu về tốc độ xử lý;
- Phù hợp để viết các đoạn lệnh ngắt và chương trình trên bộ nhớ khác.

Các tính năng cơ bản của phần cứng máy

Các phần cứng nội bộ chính của một máy tính bao gồm bộ vi xử lý, bộ nhớ, và thanh ghi. Thanh ghi là thành phần xử lý lưu trữ dữ liệu. Để thực hiện một chương trình, mã thực thi và dữ liệu của chương trình được sao chép từ các thiết bị bên ngoài vào bộ nhớ trong. Các bộ vi xử lý thực hiện các lệnh của chương trình.

Các đơn vị cơ bản của lưu trữ máy tính là bit; có thể là ON (1) hoặc OFF (0).

Các bộ xử lý hỗ trợ các kích thước dữ liệu sau:

- Word: dữ liệu 2-byte
- Doubleword: dữ liệu 4-byte (32 bit)
- Quadword: dữ liệu 8-byte (64 bit)
- Double quadword: dữ liệu 16-byte (128 bit)
- Kilobyte: 1024 bytes
- Megabyte: 1.048.576 byte

B.2 Cấu trúc cơ bản của một chương trình hợp ngữ

Một chương trình hợp ngữ .asm thường gồm ít nhất 2 phần (section):

1. Phần **.text** là phần chính chứa mã nguồn thực thi của chương trình.
2. Phần **.data** là phần khai báo biến và hằng của chương trình, ví dụ ở dưới chúng ta khai báo hai biến/hằng số **msg** và **len**.

```

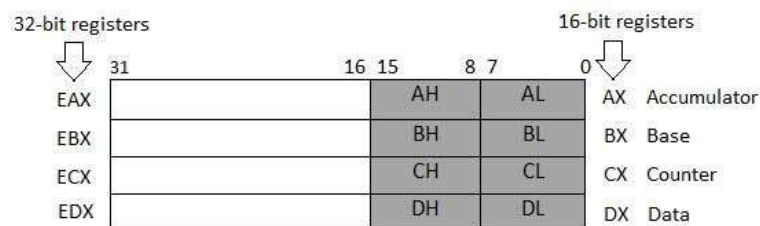
1 section .text
2     global _start      ;must be declared for using gcc
3     _start:           ;tell linker entry point
4     mov edx, len       ;message length
5     mov ecx, msg       ;message to write
6     mov ebx, 1         ;file descriptor (stdout)
7     mov eax, 4         ;system call number (sys_write)
8     int 0x80          ;call kernel
9     mov eax, 1         ;system call number (sys_exit)
10    int 0x80          ;call kernel
11
12 section .data
13
14 msg db 'Cau truc co ban cua mot chuong trinh',0xa ;our dear string
15 len equ $ - msg      ;length of our dear string
16

```

B.3 Thanh ghi, gọi hàm hệ thống và khai báo biến

- **Thanh ghi**

Khi lập trình hợp ngữ, ta có thể sử dụng các thanh ghi với các kích thước khác nhau để lưu trữ các giá trị tính toán. Hình bên dưới mô tả tên gọi và kích thước của các thanh ghi có thể sử dụng khi lập trình hợp ngữ. Ví dụ: **eax**, **ebx** là các thanh ghi 4 bytes, **al** có kích thước 1 byte và là thanh ghi con nằm bên trong **eax**.



- **Lời gọi hệ thống**

Ngoài ra, có thể sử dụng các lời gọi hệ thống (system call) để thực hiện một số tác vụ như đọc, ghi dữ liệu từ chương trình hợp ngữ. Để sử dụng, cần gán các giá trị nhất định cho 4 thanh ghi **eax**, **ebx**, **ecx** và **edx** và thực hiện lệnh gọi hệ thống **int 0x80**.

%eax	Name	%ebx	%ecx	%edx	%esx	%edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

- **Khai báo và truy xuất biến, hằng số**

Các biến và hằng số trong chương trình hợp ngữ có thể được khai báo ở section **.data**, trong đó biến có thể có các kích thước khác nhau tùy vào directive được sử dụng, còn hằng số được khai báo với từ khóa **equ**.

section .data

```
<var_name> <directive> <value>      # biến
<const_name> equ <value>            # hằng số
```

Directive	Purpose	Storage Space
DB	Define Byte	allocates 1 byte
DW	Define Word	allocates 2 bytes
DD	Define Doubleword	allocates 4 bytes
DQ	Define Quadword	allocates 8 bytes
DT	Define Ten Bytes	allocates 10 bytes

Ví dụ:

```
section .data
    msg db "Hello world"
    len equ $ - msg
    num1 dd 15
```

Ở ví dụ trên, ta có dòng khai báo biến **msg**, **num1** và hằng số **len** trong assembly. Dấu **\$** trong trường hợp này là vị trí địa chỉ (byte) ngay sau vùng địa chỉ khai báo biến **msg**, được dùng để tính toán và gán cho **len** độ dài của chuỗi **msg**.

Ngoài ra, có thể khai báo trước vùng nhớ trống có kích thước nhất định nhưng chưa có giá trị trong section **.bss** với cú pháp:

section .bss

```
<var_name> resb <size_in_bytes>
```

Trong tính toán với các lệnh hợp ngữ, hằng số chỉ có thể lấy giá trị với chính tên hằng số đó, như **len** với khai báo ở trên. Mặt khác, các biến khi khai báo trong section **.data** hoặc **.bss** sẽ được cung cấp 1 vùng nhớ và có thể truy xuất giá trị hoặc địa chỉ vùng nhớ này thông qua tên của biến. Một số cách truy xuất ô nhớ:

- **[num1]** sẽ truy xuất giá trị của ô nhớ tại vị trí **num1**, mặc định lấy 4 byte thì ta được giá trị là 15.
- **num1** sẽ tương ứng lấy địa chỉ của biến.
- **byte[msg + 5]** sẽ truy xuất 1 byte giá trị của ô nhớ nằm cách vị trí **msg** 4 byte, tức là ký tự có index là 4 trong chuỗi – ký tự 'o'.

B.4 Instruction

Các lệnh hợp ngữ rất đa dạng, cung cấp các chức năng khác nhau, từ di chuyển dữ liệu đến các phép tính toán toán học, tính toán trên bit, các lệnh rẽ nhánh. Nhìn chung, với định dạng Intel, cấu trúc chung của một lệnh hợp ngữ gồm 2 toán hạng thường là:

```
opcode dest, source
```

Trong đó, opcode sẽ khác nhau tùy vào chức năng của lệnh, dest và source có thể là các toán hạng thanh ghi, ô nhớ hoặc hằng số chứa dữ liệu cần tính toán, thao tác. Trong đó lưu ý *dest không thể là hằng số*.

- 1. Lệnh mov dest, source:** Phép gán dữ liệu từ source vào dest. Các trường hợp có thể có của lệnh mov:

```
mov register, register
mov register, immediate
mov memory, immediate
mov register, memory
mov memory, register
```

Không có lệnh mov để gán dữ liệu trực tiếp từ ô nhớ sang ô nhớ.

2. Các lệnh toán học

Lệnh	Chức năng	Ý nghĩa
add dest, source	Cộng	$dest = dest + source$
sub dest, source	Trừ	$dest = dest - source$
mul dest, source	Nhân	$dest = dest * source$
and dest, source	AND bit	$dest = dest \& source$
or dest, source	OR bit	$dest = dest source$
xor dest, source	XOR bit	$dest = dest \wedge source$
not dest	NOT bit	$dest = \sim dest$
inc dest	Tăng giá trị lên 1 đơn vị	$dest = dest + 1$
dec dest	Giảm giá trị xuống 1 đơn vị	$dest = dest - 1$
shl dest, 2	Dịch trái 2 bit	$dest = dest \ll 2$
shr dest, 2	Dịch phải 2 bit	$dest = dest \gg 2$

3. Các lệnh rẽ nhánh (Condition)

Các lệnh cmp có thể thực hiện so sánh giá trị 2 toán hạng và thay đổi 1 số cờ (flag) trong hệ thống. Sau đó, các lệnh cmp có thể được kết hợp với các lệnh rẽ nhánh có điều kiện jX để nhảy đến các đoạn mã khác nhau tùy vào kết quả so sánh.

```
cmp dest, source
jX label
```

Bảng dưới liệt kê một số lệnh jX phổ biến và điều kiện nhảy khi kết hợp với lệnh cmp.

Lệnh	Điều kiện nhảy
je	$dest == source$
jne	$dest != source$
jl	$dest < source$
jle	$dest \leq source$
jg	$dest > source$
jge	$dest \geq source$

C. THỰC HÀNH

C.1 Viết và chạy code assembly

C.1.1 Bài 1: Tìm số nhỏ nhất trong danh sách gồm N số nguyên

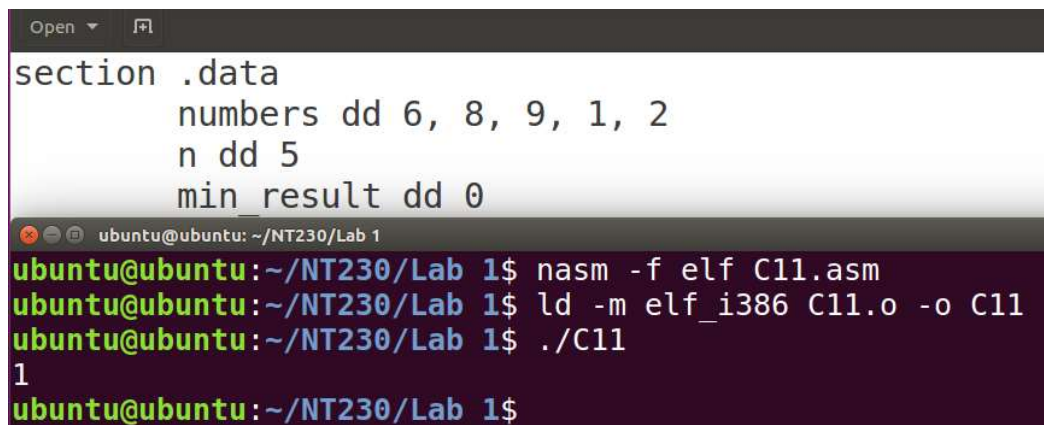
Yêu cầu 1.1: Viết chương trình assembly với các yêu cầu sau:

Input: Một mảng số nguyên gồm N số nguyên, trong đó giá trị các số và N được khai báo sẵn trong section **.data**.

Output: In ra ngoài màn hình số nhỏ nhất trong mảng.

Giới hạn: Dùng vòng lặp để duyệt qua các phần tử.

Ví dụ:



```
section .data
    numbers dd 6, 8, 9, 1, 2
    n dd 5
    min_result dd 0

ubuntu@ubuntu: ~/NT230/Lab 1
ubuntu@ubuntu:~/NT230/Lab 1$ nasm -f elf C11.asm
ubuntu@ubuntu:~/NT230/Lab 1$ ld -m elf_i386 C11.o -o C11
ubuntu@ubuntu:~/NT230/Lab 1$ ./C11
1
ubuntu@ubuntu:~/NT230/Lab 1$
```

Gợi ý 1.1: Tham khảo đoạn mã:

```
section .data
    numbers dd ?, ?, ?, ?, ? ; Array containing N integers
    n dd ? ; Number of elements in the array
    min_result dd 0 ; Result

section .text
    global _start

_start:
    ; Write code to iterate through the array to find the smallest number here

    ; Print the minimum value

    ; Exit the program
```

? Câu hỏi thêm:

1.1. Làm thế nào để tìm số lớn nhất thay vì số nhỏ nhất?

1.2. Nếu danh sách có số lượng phần tử rất lớn (hàng triệu số), làm thế nào để tối ưu hiệu suất?

C.1.2 Bài 2: Chuyển đổi số nguyên thành chuỗi HEX và in ra màn hình

Yêu cầu 1.2: Viết chương trình assembly với các yêu cầu sau:

Input: Một số nguyên được khai báo sẵn section **.data**.

Output: In ra màn hình dạng chuỗi HEX của số nguyên.

Giới hạn: Số nguyên có kích thước tối đa 4 bytes.

Ví dụ:

```
section .bss
    buffer resb 10 ; Memory to store the HEX string

section .data
    number dd 64200 ; Number to be converted
    prefix db "0x", 0 ; Prefix for HEX number
    newline db 10 ; Newline character
```

```
ubuntu@ubuntu: ~/NT230/Lab 1/C12
ubuntu@ubuntu:~/NT230/Lab 1/C12$ nasm -f elf C12.asm
ubuntu@ubuntu:~/NT230/Lab 1/C12$ ld -m elf_i386 C12.o -o C12
ubuntu@ubuntu:~/NT230/Lab 1/C12$ ./C12
0x0000FAC8
ubuntu@ubuntu:~/NT230/Lab 1/C12$ |
```

Gợi ý 2.1:

- Sử dụng **phép toán dịch bit** thay vì phép chia thông thường để tối ưu hiệu suất.
- Sử dụng phép toán AND (AND eax, 0xF) để trích từng chữ số HEX.

Gợi ý 2.2: Cần chuyển các ký tự số (nếu có) sang mã ASCII để in ra màn hình. Hiển thị kết quả ra màn hình bằng syscall.

Gợi ý 2.3: Tham khảo đoạn mã:

```
section .bss
    buffer resb 10 ; Memory to store the HEX string

section .data
    number dd ? ; Number to be converted
    prefix db "0x", 0 ; Prefix for HEX number
    newline db 10 ; Newline character

section .text
    global _start

_start:
    ; Write code to convert the number to HEX here

    ; Print to the screen

    ; Exit the program
```

? Câu hỏi thêm:

2.1. Tại sao sử dụng dịch bit (SHR) lại nhanh hơn phép chia?

2.2 Làm thế nào để mở rộng chương trình này thành nhập và chuyển đổi số bất kỳ (có dấu) thành HEX?

C.1.3 Bài 3: Đảo ngược và xét tính đối xứng của số

Yêu cầu 1.3: Viết chương trình assembly với các yêu cầu sau:

Input: Nhập một số nguyên bất kỳ.

Output: In ra màn hình YES nếu số đối xứng, NO nếu không đối xứng.

Gợi ý 3.1: Tham khảo đoạn mã:

```
section .data
    number dd ? ; Number to be checked
    result_yes db "YES", 10
    result_no db "NO", 10

section .text
    global _start
_start:
    ; Write code to check if the number is a palindrome here

    ; Print the result to the screen

    ; Exit the program
```

? Câu hỏi thêm:

3.1. Nếu nhập vào một số **rất lớn**, chương trình có gặp vấn đề gì không?

C.1.4 Bài 4: Mã hóa XOR và giải mã chuỗi

Yêu cầu 1.4: Viết chương trình assembly với các yêu cầu sau:

Input: Nhập một chuỗi ký tự được nhập từ bàn phím, một khóa **x** kích thước 1 byte khai báo sẵn trong section .data.

Output: In ra chuỗi ký tự sau khi được mã hóa XOR và chuỗi sau khi giải mã với khóa.

Gợi ý 4.1:

- Duyệt từng ký tự trong chuỗi và thực hiện **XOR** với khóa.
- Dùng **vòng lặp** để xử lý toàn bộ chuỗi.
- Dùng syscall để in kết quả ra màn hình.

? Câu hỏi thêm:

4.1. Nếu **không biết khóa**, có thể giải mã chuỗi không?

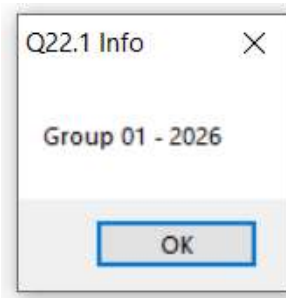
4.2. Tại sao XOR là phương pháp phổ biến trong mã độc?

C.2 Chèn mã thực thi vào file PE

C.2.1 Chèn đoạn mã hiển thị message box vào tệp Notepad.exe

Yêu cầu 2.1: Thực hiện các bước theo hướng dẫn bên dưới để chèn 1 đoạn mã vào file PE **notepad.exe** sao cho khi thực thi file sẽ hiển thị message box với nội dung **"Group X – 2026"** và tiêu đề **"Q22.1 Info"**

Ví dụ: Message box cần hiển thị:



Phần hướng dẫn dưới đây thực hiện hiển thị message box khi chạy notepad.exe:

- **Bước 1.** Mở CFF Explorer để đọc nội dung của notepad.exe và tìm kiếm các thông tin cần thiết bao gồm:
 - **AddressOfEntryPoint:** offset của vị trí thực thi code đầu tiên trong file PE khi file được chạy. Có thể tìm thấy trong phần Optional Header.
 - **ImageBase:** địa chỉ bắt đầu load file PE khi chạy.

Cộng 2 giá trị này ta có thể xác định được địa chỉ ảo (virtual address) của vị trí thực thi đầu tiên của chương trình. Ví dụ với EntryPoint là 0x0000739D và Image Base 0x01000000, virtual address của dòng code đầu được thực thi sẽ là 0x0100739D.

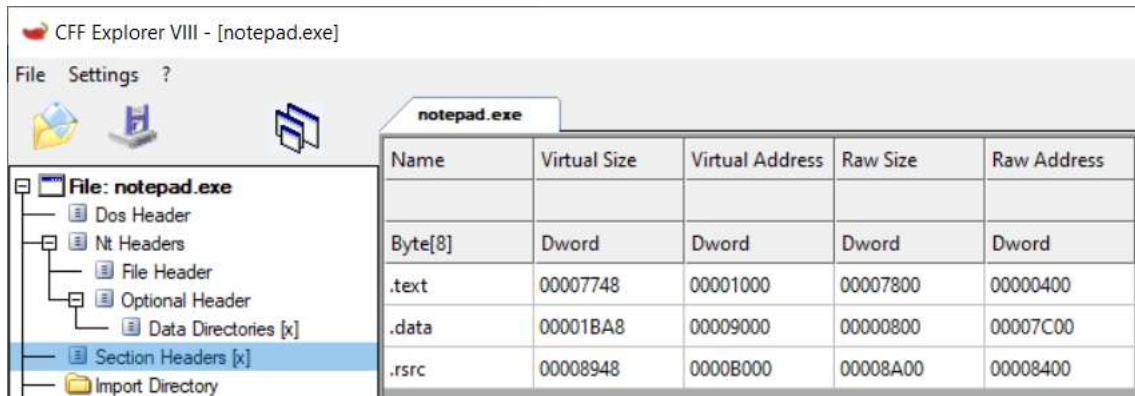
CFF Explorer VIII - [notepad.exe]

Member	Offset	Size	Value	Meaning
Magic	000000F8	Word	010B	PE32
MajorLinkerVersion	000000FA	Byte	07	
MinorLinkerVersion	000000FB	Byte	0A	
SizeOfCode	000000FC	Dword	00007800	
SizeOfInitializedData	00000100	Dword	0000A600	
SizeOfUninitializedData	00000104	Dword	00000000	
AddressOfEntryPoint	00000108	Dword	0000739D	.text
BaseOfCode	0000010C	Dword	00001000	
BaseOfData	00000110	Dword	00009000	
ImageBase	00000114	Dword	01000000	
SectionAlignment	00000118	Dword	00001000	
FileAlignment	0000011C	Dword	00000200	
MajorOperatingSystemVers...	00000120	Word	0005	

Tiếp đến chọn Section Header để quan sát các section có trong tệp PE, ta thấy 3 sections: text, data và rsrc. Mỗi section có chứa 4 thông tin:

- VirtualSize: kích thước của section khi được load vào bộ nhớ.
- VirtualAddress (VA): địa chỉ của section khi được load vào bộ nhớ.
- RawSize: kích thước của section trong PE file.
- RawAddress (RA): địa chỉ của section trong PE file.

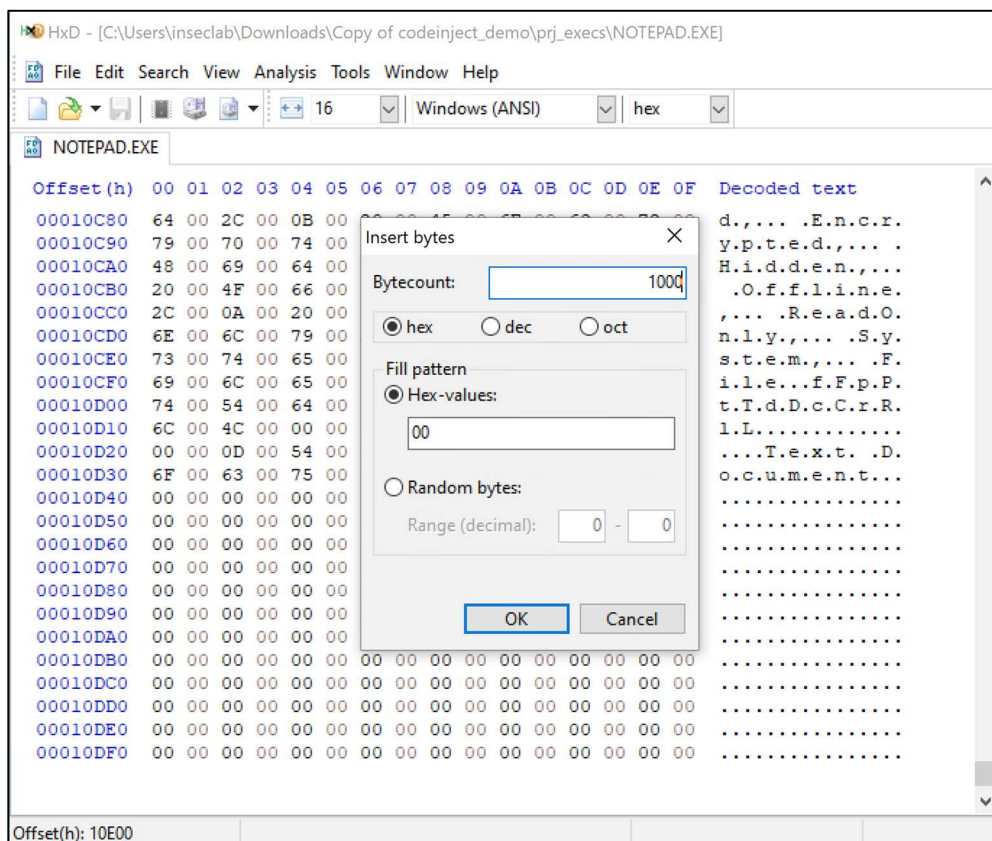
Để đơn giản, chúng ta sẽ mở rộng rsrc và chèn code vào đây.



Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.text	00007748	00001000	00007800	00000400
.data	00001BA8	00009000	00000800	00007C00
.rsrc	00008948	0000B000	00008A00	00008400

• Bước 2. Tạo vùng nhớ trong tệp tin PE

Ở bước này ta sẽ chèn thêm một số byte trống vào cuối section rsrc để tạo không gian chèn mã thực thi vào. Sử dụng công cụ HxD để mở *notepad.exe*. **Đặt trỏ chuột vào cuối file**, chọn **Edit -> Insert Bytes**, nhập giá trị *0x1000* (có thể chèn nhiều hơn đối với các đoạn mã phức tạp, tuy nhiên sẽ làm tăng kích thước của PE file) và nhấn OK.



• Bước 3. Tạo chương trình cần chèn

Ta viết một chương trình C hiển thị MessageBox như sau:

```
#include <windows.h>
int main(int argc, char * argv[])
{
    MessageBox(NULL, "Info", "Code injected", MB_OK);
    return 0;
}
```

Sau khi biên dịch và xem các mã hex ứng với các lệnh, về cơ bản, đoạn mã gọi MessageBox gồm 5 dòng lệnh với các mã hex tương ứng phía sau.

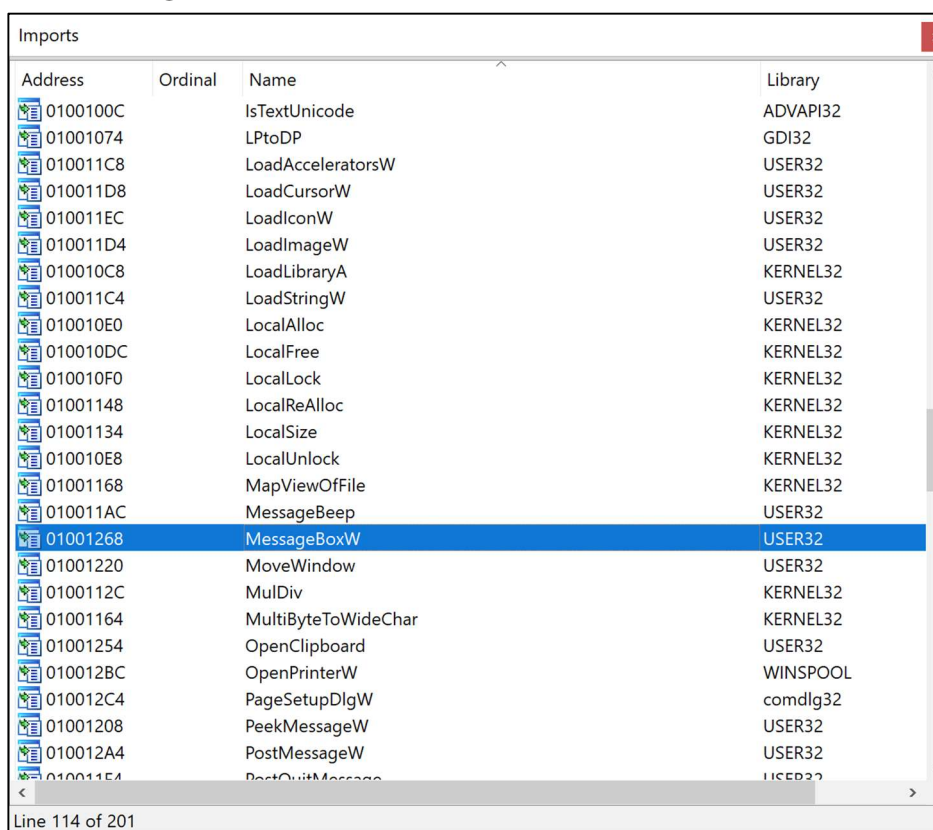
```
push 0                ; 6a 00
push Caption          ; 68 X
push Text              ; 68 Y
push 0                ; 6a 00
call [MessageBoxW]    ; ff15 Z
```

Đoạn mã cần chèn vào notepad.exe là các mã hex ứng với lệnh, tuy nhiên cần tìm các giá trị (X, Y, Z) phù hợp.

? Câu hỏi: X hay Y liên quan đến việc in được Group X và tiêu đề trong message box?

1. Giá trị **Z**: được dùng trong lệnh call gọi hàm *MessageBoxW*, chính là địa chỉ của hàm *MessageBoxW* được import từ thư viện USER32.dll.

Trong IDA Pro, mở *notepad.exe*, chọn **View -> Open Subviews -> Imports** và ta thấy địa chỉ của hàm *MessageBoxW* chính là 0x01001268.

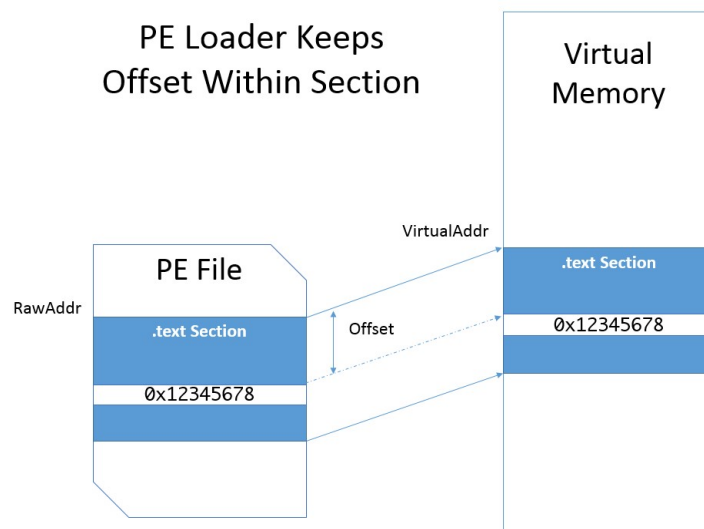


2. Các giá trị X và Y là địa chỉ của một số chuỗi cần dùng khi hiển thị message box khi file PE thực thi. Ta sẽ chèn trước các chuỗi này vào file PE và tính toán X, Y tương ứng là các địa chỉ ảo của các chuỗi.

Trong HxD, ta chọn vị trí 0x00011040 của tệp PE để lưu trữ Caption và 0x00011060 để lưu trữ Text. Ta có thể tùy chọn những vị trí khác tùy thích. Các vị trí này là offset so với vị trí bắt đầu của file PE.

Đối với mỗi section, loader sẽ sao chép section tại RA trong PE file sang bộ nhớ ảo tại VA trong khi vẫn giữ đúng offset bên trong section đó.

$$\text{Offset} = \text{RA} - \text{Section RA} = \text{VA} - \text{Section VA} \quad (1)$$



Như vậy, với các thông tin của section .rsrc, với Section RA = 0x00008400 và Section VA = 0x000B000. Với Caption ở vị trí 0x00011040 trong file PE, ta xác định VA của Caption thông qua công thức (1) như sau:

$$0x00011040 - 0x00008400 = X - 0x000B000$$

$$\rightarrow X = 0x00013C40$$

Cộng thêm ImageBase, suy ra X = 0x01013C40. Tương tự, Y = 0x01013C60.

• Bước 4. Xác định entry point mới

Ta cũng chọn địa chỉ 0x00011000 trong vùng nhớ đã được mở rộng (bước 2) để lưu trữ mã hợp ngữ. Như vậy, với cùng cách tính ở công thức (1), vị trí offset của đoạn mã này trong PE được tính như bên dưới và cũng được gán thành entry point mới của file PE:

$$\text{new_entry_point} = 0x00011000 - 0x00008400 + 0x000B000 = 0x00013C00$$

• Bước 5. Thiết lập lệnh quay về AddressOfEntryPoint ban đầu

Để chương trình *notepad.exe* tiếp tục được thực thi sau khi đã chạy đoạn code trên, ta cần chèn dòng lệnh quay về *AddressOfEntryPoint* cũ ngay sau đoạn code ở bước 3 với lệnh **jmp relative_VA**

Đối với lệnh `jmp`, đích đến (`old_entry_point`) sẽ được tính bằng cách cộng giá trị `relative_VA` vào thanh ghi PC khi lệnh được thực thi. Bởi vì PC luôn trỏ đến vị trí đầu của câu lệnh kế tiếp, cho nên cần phải tính 5 bytes của câu lệnh `jmp` nữa. Ta có công thức sau:

$$\text{old_entry_point} = \text{jmp_instruction_VA} + 5 + \text{relative_VA} \quad (2)$$

Đoạn lệnh được chèn thêm bắt đầu từ vị trí `VA = 0x01013C00`. Nếu đặt lệnh `jmp` sau 5 câu lệnh ở bước 3 gồm 20 bytes (`0x14`) thì `jmp_instruction_VA = 0x01013C14`. Với `old_entry_point = 0x0100739D`, chính là giá trị `AddressOfEntryPoint` ban đầu đã cộng `ImageBase`, ta được `relative_VA = 0x0100739D - 5 - 0x01013C14 = 0xFFFF3784`.

Đến đây, ta đã có một đoạn mã hợp ngữ hoàn chỉnh để chèn vào `notepad.exe`. Các địa chỉ được biểu diễn theo dạng little endian (x86).

```
push 0           ; 6a 00
push Caption     ; 68 403C0101
push Text        ; 68 603C0101
push 0           ; 6a 00
call [MessageBoxW] ; ff15 68120001
jmp Originl_Entry_Point ; e9 8437FFFF
```

• Bước 6. Chèn mã và chuỗi vào notepad.exe

Sử dụng HxD để chèn đoạn mã cùng với giá trị `Caption` và `Text` vào `notepad.exe`. Lưu lại file.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00010FF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011000	6A	00	68	60	3C	01	01	68	40	3C	01	01	6A	00	FF	15	j.h`<..h@<..j.ý.
00011010	68	12	00	01	E9	84	37	FF	FF	00	00	00	00	00	00	00	h...é„7ýý.....
00011020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011040	43	00	6F	00	64	00	65	00	20	00	69	00	6E	00	6A	00	C.o.d.e. .i.n.j.
00011050	65	00	63	00	74	00	65	00	64	00	00	00	00	00	00	00	e.c.t.e.d.....
00011060	49	00	6E	00	66	00	6F	00	00	00	00	00	00	00	00	00	I.n.f.o...0.....
00011070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000110A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

• Bước 7. Hiệu chỉnh các tham số trong PE header

Sử dụng CFF Explorer để thay đổi các giá trị sau:

- Trong Section Headers, thay đổi kích thước section `.rsrc`.

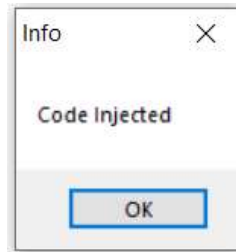
notepad.exe				
Name	Virtual Size	Virtual Address	Raw Size	Raw Address
00000228	00000230	00000234	00000238	0000023C
Byte[8]	Dword	Dword	Dword	Dword
.text	00007748	00001000	00007800	00000400
.data	00001BA8	00009000	00000800	00007C00
.rsrc	00009948	0000B000	00009A00	00008400

- Trong Optional Headers, tăng SizeOfImage lên 0x1000.
- Trong Optional Headers, chỉnh sửa AddressOfEntryPoint thành 0x00013C00.

Sau đó lưu lại file.

• Bước 8. Kiểm tra kết quả

Khi thực thi notepad.exe, một cửa sổ xuất hiện như sau:



Sinh viên thực hiện tương tự và báo cáo kết quả chèn message box hiển thị “Group X – 2026” với tiêu đề “Q22.1 Info”.

C.2.2 Cải tiến cách chèn mã thực thi

Yêu cầu 2.2: Bằng cách không tạo thêm vùng nhớ mở rộng vào tập tin PE, tận dụng vùng nhớ trống để chèn chương trình cần chèn trên tập tin Notepad và calc. Báo cáo phương pháp và kết quả thực hiện.

C.2.3 Bài tập nâng cao

Yêu cầu 2.3: Chỉnh sửa luồng thực thi mà không thay đổi Entry Point.

Thay vì thay đổi trực tiếp **AddressOfEntryPoint**, sinh viên cần phải:

- **Xác định một function có sẵn** trong Notepad.exe mà được gọi sớm (ví dụ WinMain hoặc một hàm khởi tạo).
- **Chèn đoạn code mong muốn vào giữa một function hợp lệ** bằng cách thay đổi một số lệnh **không quan trọng** để chuyển hướng thực thi.
- Đảm bảo **trả lại quyền điều khiển** cho chương trình gốc sau khi payload đã chạy.

Gợi ý:

- Dùng **IDA Pro** để phân tích mã Assembly của Notepad.exe và tìm function phù hợp.
- **Thay thế một chuỗi lệnh không quan trọng** bằng jmp hoặc call đến shellcode.
- Tránh thay đổi **bảng nhập xuất API** để giảm nguy cơ phát hiện.

Yêu cầu 2.4: Hook API WriteFile hoặc ReadFile.

Yêu cầu:

- Xác định **API WriteFile** hoặc **ReadFile** trong Notepad.exe.
- **Chèn một đoạn mã hook** để intercept các lệnh ghi file trước khi tiếp tục thực thi.

- **Ghi log dữ liệu** được ghi vào file trước khi cho phép Notepad tiếp tục.

Gợi ý:

- Dùng **IAT Hooking**: thay đổi địa chỉ của WriteFile trong bảng IAT thành shellcode.
- Dùng **Inline Hook**: ghi đè 5 byte đầu của WriteFile bằng một jmp đến shellcode.
- Đảm bảo **khôi phục lại hook** để Notepad không bị crash.

Yêu cầu 2.5: Mã hóa Shellcode bằng XOR..

Thay vì **chèn shellcode trực tiếp**, bạn phải:

- **Mã hóa shellcode bằng thuật toán XOR** với một khóa đơn giản (ví dụ 0x55).
- **Chỉ chèn shellcode đã mã hóa vào Notepad.exe.**
- **Viết một đoạn code nhỏ để giải mã shellcode** trước khi thực thi.

Gợi ý:

- **Duyệt qua từng byte của shellcode và XOR với 0x55** trước khi lưu vào PE.
- Dùng một **stub nhỏ để giải mã shellcode ngay trong bộ nhớ.**
- Không lưu khóa XOR dưới dạng **hằng số trong mã nguồn** để tránh bị phát hiện.

Yêu cầu 2.6: Che giấu dấu vết trong Header PE.

Để tránh bị phát hiện, yêu cầu:

- **Chỉnh sửa các giá trị trong PE Header** để làm cho file trông hợp lệ hơn.
- **Thay đổi TimeDateStamp, SizeOfImage, hoặc Section Names** để mô phỏng một tập tin hợp pháp.
- **Cập nhật lại checksum của tập tin PE** sau khi chỉnh sửa.

Gợi ý:

- Dùng **CFF Explorer** để thay đổi TimeDateStamp về một giá trị ngẫu nhiên.
- **Giả mạo một tập tin PE hợp lệ** bằng cách đổi tên các Section .text → .xyz1.
- **Tăng nhẹ kích thước tập tin** để tránh bị phát hiện bởi hash-based detection.

Yêu cầu 2.7: Mã tự thay đổi (Self-Modifying Code).

Yêu cầu:

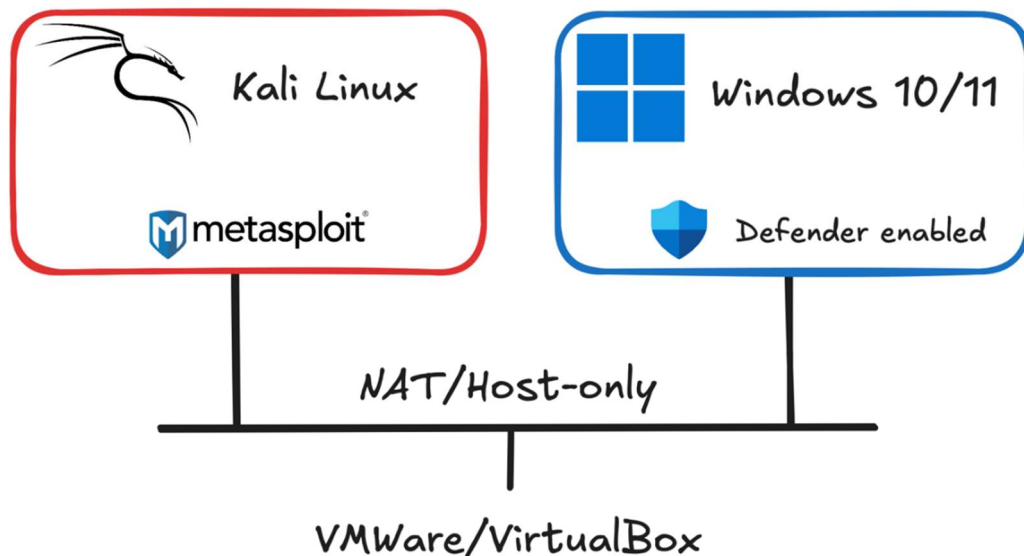
- **Viết shellcode có thể tự sửa đổi chính nó trong bộ nhớ** trước khi chạy.
- Thực hiện kỹ thuật **“polymorphic shellcode”** để thay đổi opcode tại runtime.
- **Không sử dụng jmp/call trực tiếp**, mà thay vào đó sử dụng **stack pivoting**.

Gợi ý:

- Dùng **VirtualProtect** hoặc **VirtualAlloc** để làm shellcode có thể ghi.
- **Tạo một vòng lặp đơn giản để thay đổi chính opcode của shellcode** trước khi thực thi.
- Tận dụng **“Return-Oriented Programming” (ROP) gadgets** để tránh dùng trực tiếp jmp/call.

Yêu cầu 2.8: Tấn công khai thác lỗ hổng trong môi trường mạng nội bộ.

Yêu cầu: Sinh viên sẽ triển khai một mô hình mạng nội bộ **mô phỏng kịch bản tấn công thực tế**, sử dụng kỹ thuật **shellcode injection** và **persistence** để duy trì quyền truy cập từ xa vào hệ thống mục tiêu.



Mô hình hệ thống

- **Máy A:** Cài đặt **Metasploit Framework** (Kali Linux, Parrot OS hoặc Windows có Metasploit).
- **Máy B:** Chạy **Windows 10/11 (bản cập nhật mới nhất)** với **Windows Defender** được bật.
- **Mạng:** Máy A và B **cùng trong mạng nội bộ** (có thể sử dụng **VMware** hoặc **VirtualBox** để tạo mạng ảo nội bộ).

Nhiệm vụ:

10.1. Reverse Shell Payload Execution

- Từ máy A, sử dụng msfvenom để tạo **Reverse HTTP Shellcode**.
- **Viết chương trình C/C++ trên máy B** để thực thi shellcode này, sao cho khi chạy chương trình, **máy B sẽ kết nối ngược (reverse shell) về máy A**.
- **Thiết lập persistence:** Tìm cách **tự động thực thi chương trình** khi máy B khởi động lại. **Tối thiểu 3 cách**.

? Câu hỏi thêm:

- Nếu Windows Defender phát hiện shellcode, làm sao để bypass?
- Làm thế nào để thực thi shellcode mà không lưu file .exe trên ổ đĩa?

10.2 Shellcode Injection vào Ứng Dụng Hợp Pháp (WinRAR)

- Từ máy A, tạo **Reverse Shell Payload** giống như bài 10.1.
- **Tìm cách chèn shellcode vào chương trình WinRAR** trên máy B.

- **Khi người dùng mở WinRAR, shellcode sẽ tự động được thực thi** và kết nối về máy A.
- **Lưu ý:** Mọi thao tác trên máy B phải thực hiện **dưới quyền user bình thường**, không có quyền Administrator.

Gợi ý:

Bước 1. Injector quét tiến trình, nếu thấy có WinRAR đang chạy, chèn shellcode vào bộ nhớ của tiến trình, đồng thời tạo một luồng thực thi (thread) mới để chạy shellcode ngay bên trong WinRAR.

Bước 2. Thread vừa được tạo trong WinRAR sẽ kích hoạt kết nối ngược về C2, báo hiệu rằng shellcode đã được thực thi thành công.

Bước 3. Máy chủ C2 tiếp tục gửi thêm payload bổ sung để mở rộng quyền kiểm soát trên WinRAR.

Bước 4. WinRAR nhận đủ payload cần thiết và bắt đầu giao tiếp với C2, tạo điều kiện cho kẻ tấn công kiểm soát hệ thống mà không gây ảnh hưởng đến hoạt động bình thường của ứng dụng.

? Câu hỏi thêm:

- Làm thế nào để shellcode hoạt động mà không làm WinRAR bị lỗi?
- Có cách nào ẩn shellcode để tránh bị Windows Defender phát hiện không?

D. YÊU CẦU & ĐÁNH GIÁ

- Sinh viên tìm hiểu và thực hành theo hướng dẫn, thực hiện **theo nhóm đã đăng ký**.
- Nộp báo cáo kết quả gồm **Code, File được export** và chi tiết những việc (**Report**) mà nhóm đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích cho quan sát (nếu có).
- **Báo cáo:**
 - File **.PDF**. Tập trung vào nội dung, không mô tả lý thuyết.
 - Đặt tên theo định dạng: **[Mã lớp]-LabX_MSSV1-MSSV2-MSSV3**.
Ví dụ: [NT230.Q2X.ANTT]-Lab1_2352xxxx-2352yyyy-2352zzzz.
 - Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
 - Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Bài sao chép, trễ, ... sẽ được xử lý tùy mức độ vi phạm.

HẾT

Chúc các bạn hoàn thành tốt!